

# LAPORAN PRAKTIKUM JOBSHEET 6

## ALGORITMA DAN STRUKTUR DATA: SEARCHING

Nama : Dhafir Tsabit  
NIM : 254107020148  
No. Absen : 11  
Kelas : TI-1C

### 1. TUJUAN PRAKTIKUM

Setelah melakukan materi praktikum ini, mahasiswa mampu:

1. Menjelaskan mengenai algoritma Searching.
2. Membuat dan mendeklarasikan struktur algoritma Searching.
3. Menerapkan dan mengimplementasikan algoritma Searching.

### 2. Percobaan 1 — Searching/ Pencarian Menggunakan Algoritma Sequential Search

#### 2.1 Kode Program

Kode program Percobaan 1 dapat dilihat melalui repository pada tautan berikut:

<https://github.com/dhafirtibast/PraktikumASD/tree/main/Jobsheet5>

#### 2.2 Verifikasi Hasil

|                                                                                                                                                                                                                                                                          |                                                                                                                                                                                                                                                                      |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>Masukkan kapasitas queue: 4 Masukkan operasi yang diinginkan: 1. Enqueue 2. Dequeue 3. Print 4. Peek 5. Clear ----- 1 Masukkan data baru: 15 Masukkan operasi yang diinginkan: 1. Enqueue 2. Dequeue 3. Print 4. Peek 5. Clear ----- 1 Masukkan data baru: 31</pre> | <pre>Masukkan operasi yang diinginkan: 1. Enqueue 2. Dequeue 3. Print 4. Peek 5. Clear ----- 4 Elemen terdepan: 15 Masukkan operasi yang diinginkan: 1. Enqueue 2. Dequeue 3. Print 4. Peek 5. Clear ----- 0 PS C:\Users\DHAFIR\Documents\ASD\P1Jobsheet10&gt;</pre> |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

#### 2.3 Jawaban Pertanyaan Percobaan 1

1. Perbedaan antara method tampilDataSearch dan tampilPosisi:
  - o tampilPosisi berfungsi untuk menginformasikan letak indeks atau posisi elemen yang dicari dalam array jika data ditemukan, atau memberikan pesan bahwa data tidak ditemukan.

- tampilDataSearch berfungsi untuk menampilkan seluruh pasang atribut (detail informasi objek seperti NIM, Nama, Kelas, dan IPK) dari elemen yang berada di posisi indeks hasil pencarian tersebut.
- 2. Fungsi perintah break pada potongan kode program: Perintah break berfungsi untuk langsung menghentikan proses perulangan (for) begitu data yang dicari pertama kali ditemukan. Hal ini dilakukan demi efisiensi program agar tidak perlu melanjutkan pencarian ke sisa elemen array berikutnya setelah target terpenuhi.
- 3. Fungsi variabel pos atau indeks hasil pencarian: Variabel pos berfungsi sebagai penanda status dan lokasi hasil pencarian. Jika bernilai -1, berarti data tidak ditemukan. Jika bernilai selain -1, variabel ini menyimpan posisi indeks tepat di mana objek yang dicari berada, yang nantinya digunakan sebagai acuan untuk menampilkan data spesifik atau posisi koordinatnya.
- 4. Hasil pencarian jika terdapat lebih dari satu data dengan nilai yang sama: Program akan menampilkan data yang pertama kali ditemukan (indeks terkecil). Hal ini dikarenakan adanya perintah break yang langsung menghentikan struktur perulangan sesaat setelah kondisi `if (listMhs[j].ipk == cari)` terpenuhi untuk pertama kalinya.
- 5. Apa yang terjadi jika perintah break dihapus: Jika perintah break dihapus, perulangan akan terus berjalan menyisir seluruh isi array hingga indeks terakhir (`listMhs.length - 1`). Dampaknya, apabila ada beberapa data dengan IPK sama, variabel posisi akan terus diperbarui dan hasil akhirnya akan menyimpan serta menampilkan posisi data yang paling terakhir ditemukan (indeks terbesar).

### 3. Percobaan 2 — Searching/ Pencarian Menggunakan Algoritma Binary Search

#### 3.1 Kode Program

Kode program Percobaan 2 dapat dilihat melalui repository pada tautan berikut:

<https://github.com/dhafirtibast/PraktikumASD/tree/main/Jobsheet5>

#### 3.2 Verifikasi Hasil

#### 3.3 Jawaban Pertanyaan Percobaan 2

1. Proses divide:

```
if (right >= left) {
    mid = (left + right) / 2;
```

2. Proses conquer:

```
else if (listMhs[mid].ipk > cari) {
    return findBinarySearch(cari, left, mid-1);
}
else {
    return findBinarySearch(cari, mid+1, right);
}
```

3. Fungsi left, right, dan mid:

- left: Menandakan batas indeks terluar sebelah kiri (awal) dari sub-array yang sedang diperiksa.

- right: Menandakan batas indeks terluar sebelah kanan (akhir) dari sub-array yang sedang diperiksa.

- mid: Menandakan posisi indeks nilai tengah yang membagi dua sub-array sekaligus menjadi titik evaluasi nilai target.

4. Apakah program tetap berjalan jika input IPK tidakurut? Mengapa? Program akan tetap berjalan (tidak error secara sistem), namun hasil pencariannya bisa menjadi tidak valid atau tidak ditemukan (salah). Hal ini terjadi karena algoritma *Binary Search* bekerja dengan asumsi bahwa data sudah dalam keadaan terurut agar dapat mengeliminasi setengah bagian data secara konsisten. Jika acak, keputusan untuk bergeser ke kiri atau ke kanan menjadi keliru.
5. Hasilnya tidak akan sesuai atau memicu status tidak ditemukan, karena logika bawaannya dirancang untuk data *ascending*, `listMhs[mid].ipk > cari` akan mengarah ke kiri.

Modifikasi kode:

```
//Modif descending  
else if (listMhs[mid].ipk < cari)
```

6. Pencarian dinyatakan gagal saat kondisi rekursif `if (right >= left)` bernilai false atau saat nilai `left > right`. Kondisi ini menandakan bahwa ruang pencarian telah menyempit hingga habis dan saling bersilangan tanpa menemukan kecocokan nilai, sehingga method mengembalikan nilai '-1'.
7. Modifikasi input jumlah mahasiswa:

```
System.out.print(s: "Masukkan jumlah mahasiswa: ");  
int jumMhs=sc.nextInt();  
sc.nextLine();  
  
MahasiswaBerprestasi11 list = new MahasiswaBerprestasi11(jumMhs);
```